

Tarea C — Recuperación léxica contra semántica latente sobre un corpus mínimo

MMIA 6013 · Taller 03 v2 · Entrega: `reporte.pdf` (máx. 2 páginas, con código)

Objetivo

Comparar, sobre un corpus de 10 documentos y 6 consultas con un documento relevante cada una (tarea-c-recuperacion-lexica-lsa.pdf), dos recuperadores: (1) la línea base léxica TF-IDF (`TfidfVectorizer` por defecto) con similitud coseno y (2) la semántica latente LSA (`TruncatedSVD` a 4 dimensiones) sobre la misma matriz TF-IDF. Se evalúan `Hit@1`, `Hit@3` y `MRR`, se analizan las consultas falladas mirando los términos que comparten la consulta y su documento relevante, y se discute por qué un recuperador denso resolvería mejor `q5` y qué costo tiene frente a TF-IDF.

Método

Los datos están codificados en el script (sin descargas): corpus `d01–d10` y consultas `q1–q6` con relevantes `d04`, `d03`, `d08`, `d05`, `d09`, `d02`. El pipeline:

1. **TF-IDF:** `TfidfVectorizer()` con parámetros por defecto, ajustado solo con los 10 documentos (vocabulario: 105 términos); las consultas se transforman con el mismo vectorizador; similitud coseno consulta–documento y orden descendente.
2. **LSA:** `TruncatedSVD(n_components=4, random_state=0)` ajustado sobre la matriz TF-IDF de documentos; las consultas se proyectan con el mismo SVD; similitud coseno en el espacio latente de 4 dimensiones. Varianza explicada por las 4 componentes: 0.39534217090869533.
3. **Métricas** sobre las 6 consultas: `Hit@1` (relevante en el puesto 1), `Hit@3` (entre los 3 primeros) y `MRR` (media de $1/\text{rango del relevante}$). Criterio de fallo (Parte 3): relevante fuera del top 3.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

vec = TfidfVectorizer()                # parámetros por defecto
X = vec.fit_transform(docs)            # documentos (10 x 105)
Q = vec.transform(consultas)          # consultas, mismo vocabulario
S_tfidf = cosine_similarity(Q, X)      # ranking léxico

svd = TruncatedSVD(n_components=4, random_state=0)
X_lsa = svd.fit_transform(X)           # documentos a 4 dimensiones
Q_lsa = svd.transform(Q)               # consultas al mismo espacio
S_lsa = cosine_similarity(Q_lsa, X_lsa)

def metricas(S):                       # Hit@1, Hit@3, MRR
    ranks = [int(np.argsort(-s)[rel[i]]) + 1 for i, s in enumerate(S)]
    return (np.mean([r == 1 for r in ranks]),
            np.mean([r <= 3 for r in ranks]),
            np.mean([1 / r for r in ranks]))
```

Resultados

Tabla 1 — Parte 2: métricas de los dos métodos (6 consultas, corpus `d01–d10`).

Método	Hit@1	Hit@3	MRR
TF-IDF + coseno	0.8333	0.8333	0.8611
LSA (4 dim.) + coseno	0.6667	0.8333	0.7500

Tabla 2 — Parte 3: análisis por consulta (términos compartidos según la tokenización del vectorizador).

Consulta	Relevante	Términos compartidos consulta–relevante	Rank TF-IDF	Rank LSA
----------	-----------	---	-------------	----------

q1	d04	de, frecuencia, función, la, léxica, pondera, ranking, términos	1	1
q2	d03	al, fragmentos, prompt	1	1
q3	d08	bajo, de, entrena, matrices, rango	1	1
q4	d05	compara, la, orientación, vectores	1	3
q5	d09	la	6	6
q6	d02	autoatención, capas, con, feed, forward	1	1

Consultas falladas (relevante fuera del top 3): **TF-IDF** → **q5**; **LSA** → **q5**.

Detalle de los fallos y degradaciones:

- **q5 (relevante d09):** TF-IDF top-3 = d03 (0.3107), d06 (0.2009), d04 (0.1896); d09 queda en el puesto 6 con sim = 0.0790. LSA top-3 = d04 (0.9427), d01 (0.8066), d03 (0.6141); d09 vuelve a quedar en el puesto 6 (sim = 0.500).
- **q4 (relevante d05):** LSA degrada el relevante del puesto 1 al 3 (d01 con 0.9220 y d10 con 0.9206 superan a d05 con 0.8984); sigue dentro del top 3, por lo que no cuenta como fallo.

Discusión

Parte 3 — por qué falla cada método. El único fallo, común a ambos, es q5. Su consulta comparte con d09 un solo término, «la», sin valor discriminativo; los siete términos de contenido de la consulta — determinista, elegir, hace, más, palabra, sea, siguiente — están fuera del vocabulario TF-IDF (no aparecen en ningún documento), porque d09 expresa la misma idea con otro léxico: temperatura, logits, softmax, probabilidad. Sin solape léxico, TF-IDF solo se apoya en palabras funcionales, y d03, d06 y d04 — que comparten con la consulta términos genéricos como «modelo» o «al» — la superan. LSA no repara el fallo: con 10 documentos y 4 componentes (varianza explicada 0.39534217090869533) no hay estadística de co-ocurrencia suficiente para aprender la paráfrasis «más determinista» ↔ «valores bajos concentran la probabilidad», y el relevante queda también en el puesto 6. En las demás consultas el relevante comparte términos distintivos con la consulta (p. ej. «ranking/léxica/pondera» en q1, «matrices/bajo rango» en q3, «autoatención/feed-forward» en q6) y ambos métodos lo colocan en el puesto 1. LSA incluso perjudica a q4 (puesto 1 → 3): la proyección latente difumina emparejamientos léxicos precisos cuando el corpus es diminuto.

Parte 4 — recuperador denso para q5 y su costo. Un recuperador denso resolvería q5 mejor porque un encoder entrenado a gran escala mapea intención y paráfrasis al mismo espacio semántico: «más determinista al elegir la siguiente palabra» y «valores bajos concentran la probabilidad» quedarían como vectores cercanos aunque no compartan ninguna palabra de contenido; la recuperación densa entiende semántica, sinónimos y paráfrasis, mientras que los métodos dispersos como TF-IDF/BM25 solo coinciden si hay matching léxico y no entienden sinónimos ni paráfrasis (s2-rag-y-vector-search.md). El experimento lo evidencia: con solape nulo, ambos métodos colocan a d09 en el puesto 6 (Tabla 2). Los costos frente a TF-IDF son: (i) requiere un modelo preentrenado y cómputo de inferencia (típicamente GPU) para embeber consultas y documentos, mientras que TF-IDF no requiere entrenamiento ni recursos especiales; (ii) exige indexar y almacenar los embeddings de todo el corpus; (iii) pierde interpretabilidad, pues no hay un matching de términos inspeccionable como el de la Tabla 2; y (iv) corre riesgo de desajuste de dominio si el encoder no fue entrenado con texto similar. A cambio, TF-IDF/BM25 siguen siendo exactos con términos técnicos, siglas y nombres propios y no necesitan GPU (s2-rag-y-vector-search.md).